Software Architecture

Design Pattern 2 – Factory method, Abstract Factory

Practice Week 11

2025학년도 2학기

목요일 13:00 – 15:00

신공학관 1214호

Konkuk Univ
DMS LAB

Design Patterns

- 생성(Creational) Patterns
  - 객체를 '만드는' 방법
  - "어떤 재료(객체)를, 어떤 공장(팩토리)에서 가져올 것인가?"
    - Singleton, **Factory Method, Abstract Factory**, **Builder**, **Prototype**
- 구조(Structural) Patterns
  - 객체들을 '조립하는' 방법
  - 그 재료들(벽, 창문, 배관)을 어떻게 '조립'하여 집의 뼈대를 만들 것인가?"
    - Adaptor, Bridge, Composite, Decorator, Façade, Flyweight, Proxy
- 행동(Behavioral) Patterns
  - 객체들이 '소통하는' 방법
  - "집에 사는 가족(객체)들이 어떻게 '소통'하고 '역할'을 나눌 것인가?"
    - Observer, Strategy, Command, State, Chain of Responsibility, Visitor, Interpreter, Memento, Mediator, Template method, Iterator

Design Pattern 2

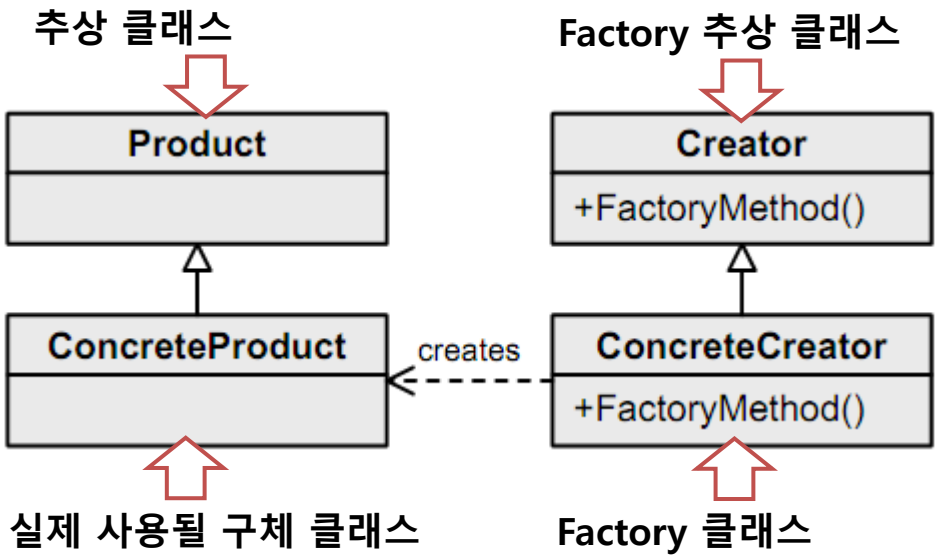
- Factory method(생성 패턴)

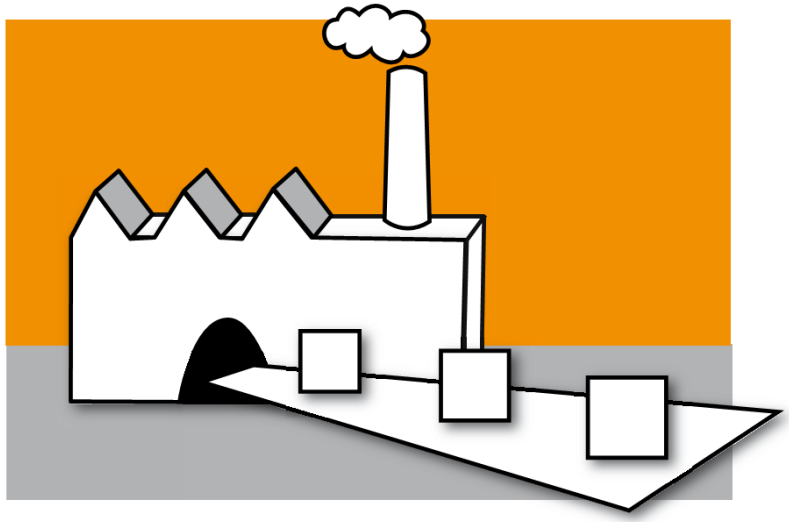
디자인 패턴 : Factory Method

Factory Method

- 어떤 "피자"를 만들지는 알아서 할게!
- Factory Method 패턴은 '**객체 생성**'이라는 작업 자체를 '**추상 메소드**'로 정의하고, '**어떤 클래스의 객체를 생성할지**'에 대한 실제 결정을 서브클래스(subclass)에게 미루는(**위임**하는) 패턴입니다.
- 어떤 객체를 만들지를 서브클래스가 정하게 하여, 부모 클래스의 코드를 변경하지 않고도 생성되는 객체의 종류를 바꿀수 있도록 함(유연성, Open Close Principle 원칙)

기본 클래스 다이어그램





Factory Patterns

- 공장
- 객체 생성을 캡슐화 하는 패턴
- 클래스로부터 생성되는 인스턴스의 형식은 실행 시 주어진 조건에 따라 결정된다.

- 예시) 피자주문 – 간단한 Factory
  - Cheese, Greek, Peperoni
  - 메뉴 변경/삭제/추가 시에 코드를 수정해야 함
  - 조건에 따라 인스턴스가 달라지는 부분을 factory class로 만들기

```
Pizza orderPizza(String type) {
    Pizza pizza;

    if (type.equals("cheese")) {
        pizza = new CheesePizza();
    } else if (type.equals("greek") {
        pizza = new GreekPizza();
    } else if (type.equals("pepperoni") {
        pizza = new PepperoniPizza();
    }

    pizza.prepare();
    pizza.bake();
    pizza.cut();
    pizza.box();
    return pizza;
}
```

orderPizza 메소드 인자로 피자 종류를 전달합니다.

피자 종류를 바탕으로 올바른 구상 클래스의 인스턴스를 만들고 pizza 인스턴스 변수에 그 인스턴스를 대입합니다. 여기에 있는 모든 피자 클래스는 Pizza 인터페이스를 구현합니다.

피자를 준비(반죽을 평평하게 펴고, 소스, 토핑, 치즈 등을 추가)한 다음 굽고, 자르고, 상자에 넣습니다. Pizza의 서브클래스(CheesePizza, VeggiePizza 등)는 피자 종류별 준비 방법을 알고 있습니다.

Factory Patterns

- 공장
- 객체 생성을 캡슐화 하는 패턴
- 클래스로부터 생성되는 인스턴스의 형식은 실행 시 주어진 조건에 따라 결정된다.

- 예시) 피자주문 – 간단한 Factory
  - Cheese, Greek, Peperoni
  - 메뉴 변경/삭제/추가 시에 코드를 수정해야 함
  - **조건에 따라 인스턴스가 달라지는 부분을 factory class로 만들기**

```
public class PizzaStore {
    SimplePizzaFactory factory;

    public PizzaStore(SimplePizzaFactory factory) {
        this.factory = factory;
    }

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = factory.createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }

    // 기타 메소드
}
```

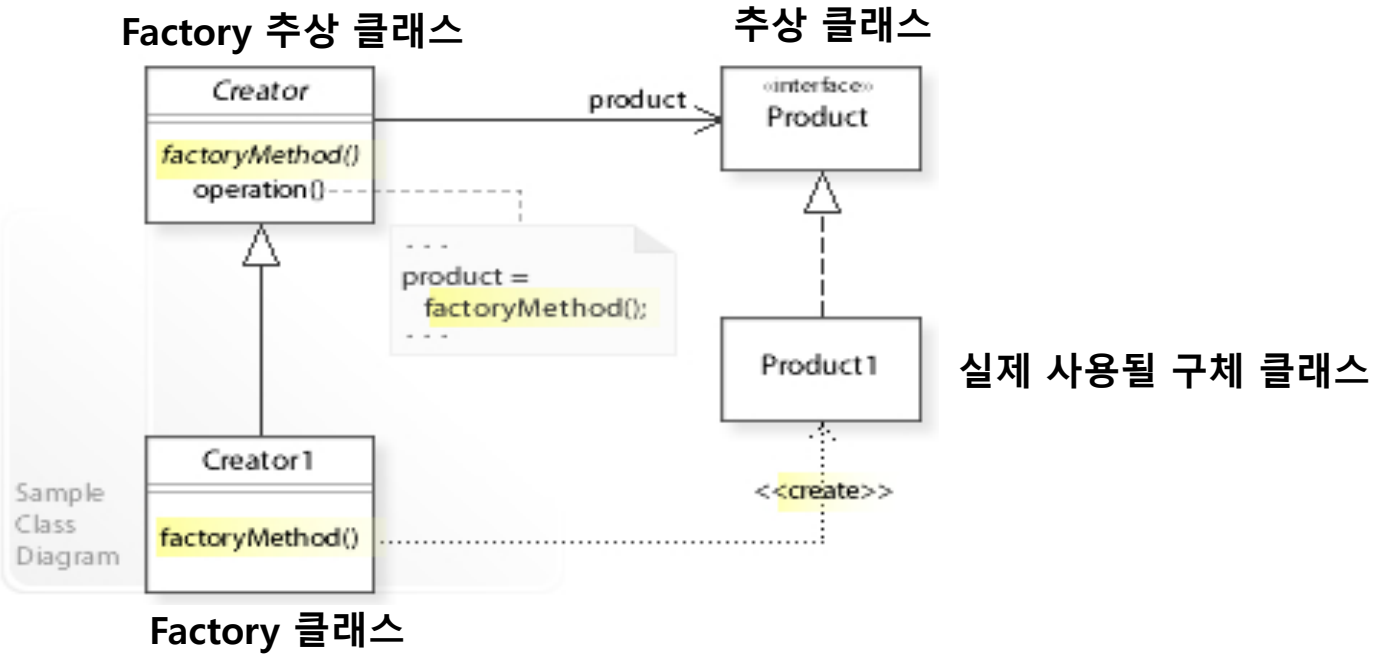
PizzaStore의 생성자에 팩토리 객체가 전달됩니다.

orderPizza( ) 메소드는 팩토리로 피자 객체를 만듭니다.
주문받은 형식을 그냥 이쪽으로 전달하기만 하면 되죠.

new 연산자 대신 팩토리 객체에 있는 create 메소드를 썼습니다.
이제 더 이상 구상 클래스의 인스턴스를 만들 필요가 없습니다!

Factory Method (팩토리 메서드)

- **주어진 입력 데이터**에 따라 **여러 개의 반환 가능한 클래스들 중 하나의 인스턴스**를 반환
- 보통 반환되는 클래스들은 공통의 부모(parent) 클래스와 공통의 메소드들을 가지고 있지만, 각각 다른 작업(task)을 수행하며 다른 종류의 데이터에 최적화
- 어떠한 서브 클래스를 생성할지 결정된 후에 실제 클래스 인스턴스가 생성됨



Factory Method (팩토리 메서드)

- 예시) 피자주문 – 가게마다 다른 pizza만드는 방법?
  - FactoryMethod(): createPizza() → NewYork Style Pizza를 만드는 factory(Store), Chicago Style Pizza
  - Product: 특정 조건에 따라 createPizza()를 사용해 만들어지는 인스턴스

이제 PizzaStore는 추상 클래스가 됩니다.
왜 그런지는 밑에서 알 수 있습니다.

```
public abstract class PizzaStore {

    public Pizza orderPizza(String type) {
        Pizza pizza;

        pizza = createPizza(type);

        pizza.prepare();
        pizza.bake();
        pizza.cut();
        pizza.box();

        return pizza;
    }

    abstract Pizza createPizza(String type);
}
```

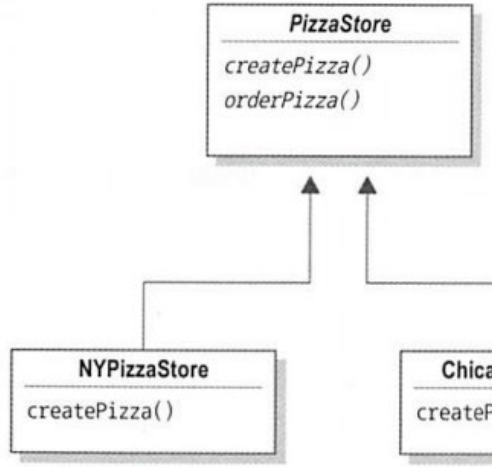
orderPizza()는 어떤 클래스에서 작업이 처리되는지 알 수 없음

팩토리 객체가 아닌 PizzaStore에 있는 createPizza를 호출합니다.

나머지는 똑같습니다.

팩토리 객체 대신 이 메소드를 사용합니다.

이제 '팩토리 메소드(factory method)'가 PizzaStore의 추상 메소드로 바뀌었습니다.



호출되는 서브클래스에 따라 createPizza()의 결과(생성되는 인스턴스)가 달라짐

```
public Pizza createPizza(type) {
    if (type.equals("cheese")) {
        pizza = new NYStyleCheesePizza();
    } else if (type.equals("pepperoni") {
        pizza = new NYStylePepperoniPizza();
    } else if (type.equals("clam") {
        pizza = new NYStyleClamPizza();
    } else if (type.equals("veggie") {
        pizza = new NYStyleVeggiePizza();
    }
}
```

```
public Pizza createPizza(type) {
    if (type.equals("cheese")) {
        pizza = new ChicagoStyleCheesePizza();
    } else if (type.equals("pepperoni") {
        pizza = new ChicagoStylePepperoniPizza();
    } else if (type.equals("clam") {
        pizza = new ChicagoStyleClamPizza();
    } else if (type.equals("veggie") {
        pizza = new ChicagoStyleVeggiePizza();
    }
}
```

Factory Method (팩토리 메서드)

- 사용법
  - **서브클래스에서 어떤 클래스를 만들지 결정**함으로써 객체 생성을 캡슐화 하는 패턴
  - 생산자(Creator, Factory) 클래스는 어떤 인스턴스를 만들지 모름.
  - 생성될 객체(Product)들의 정확한 유형을 미리 모르는 경우에 객체 생성에 필요한 전처리, 후처리 등의 과정을 미리 구성해놓고 유연하게 정할 수 있는 패턴

- 장점
  - Creator 클래스가 Product와 느슨하게 결합되어 있으므로 Product가 변경되어도 Creator의 코드를 변경할 필요가 없음.
  - 객체 생성에 있어 필요한 리소스를 Creator에 작성하여, 재사용할 수 있음.
  - Product를 유연하게 결정할 수 있음.

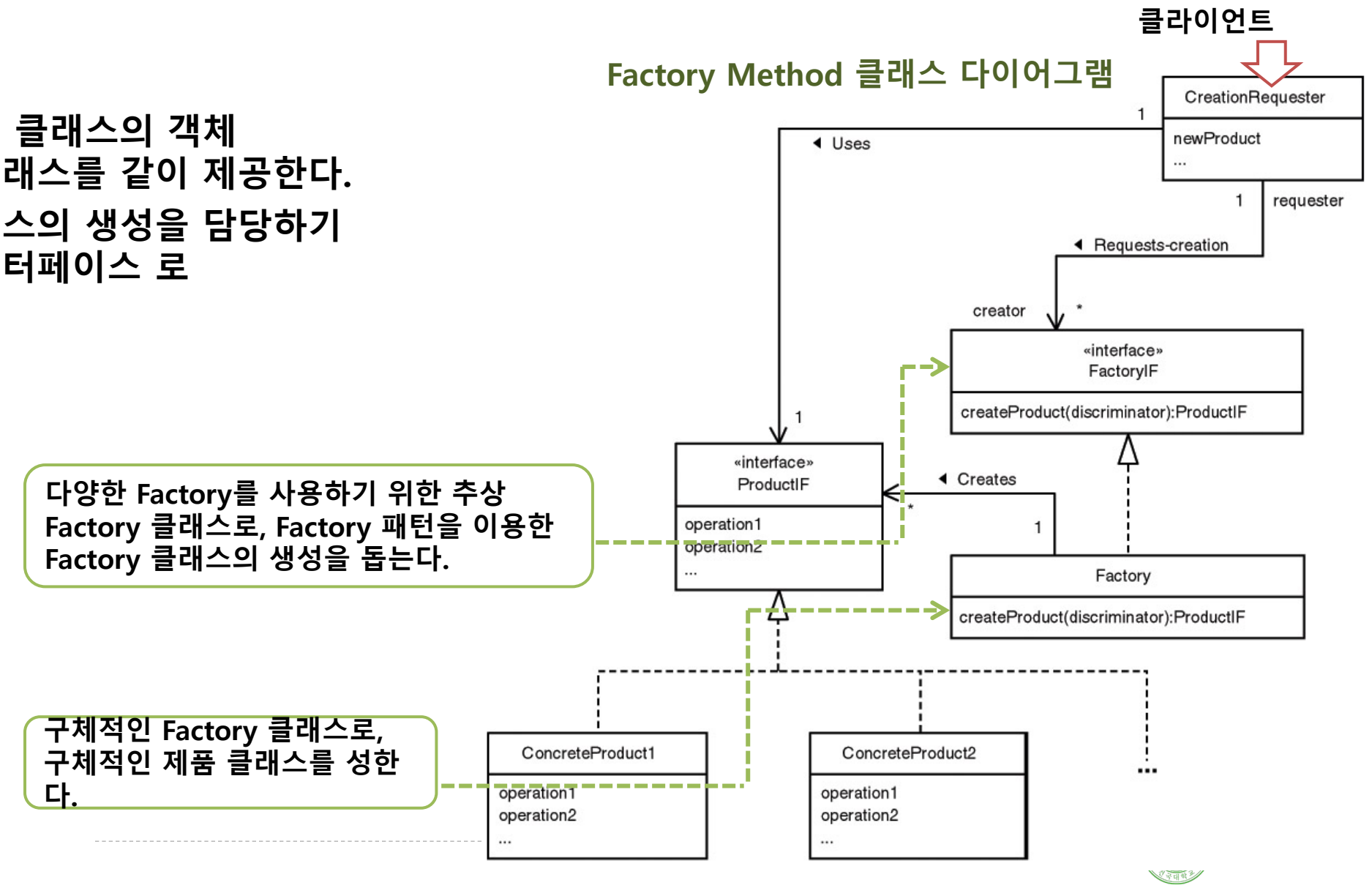
- 단점
  - 새로운 '제품' 하나를 추가할 때 마다, 그 제품을 생성하는 '생성자 서브클래스'도 하나씩 추가해야하기 때문에 클래스의 수가 많아져 코드가 복잡해질 수 있음

디자인 패턴 : Factory Method

Factory Method

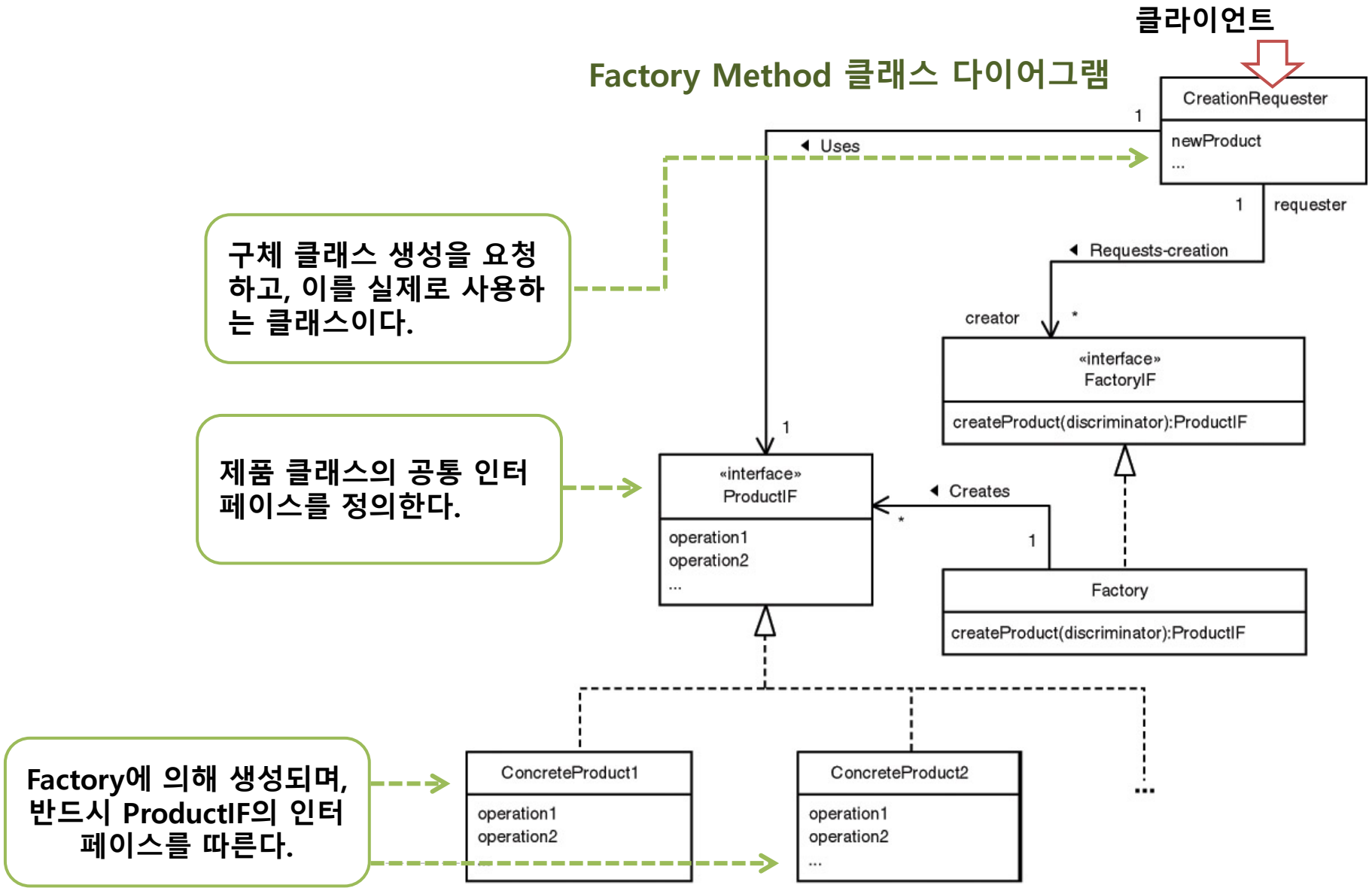
- 구체 클래스와 다른 구체 클래스의 객체 생성을 위임하는 추상 클래스를 같이 제공한다.
- 추상 클래스가 구체 클래스의 생성을 담당하기 위해서는 공통의 같은 인터페이스 로 구현되어야 한다.

Factory Method 클래스 다이어그램



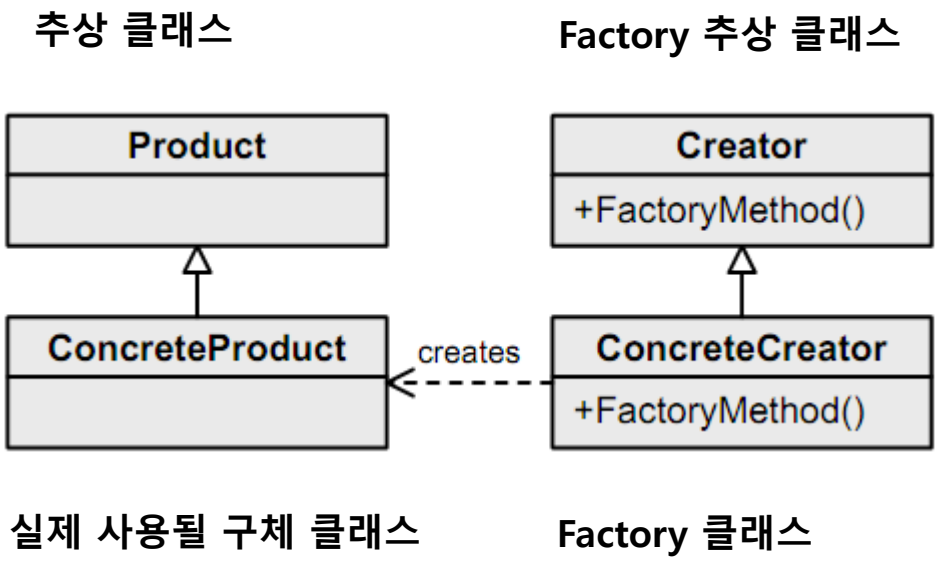
디자인 패턴 : Factory Method

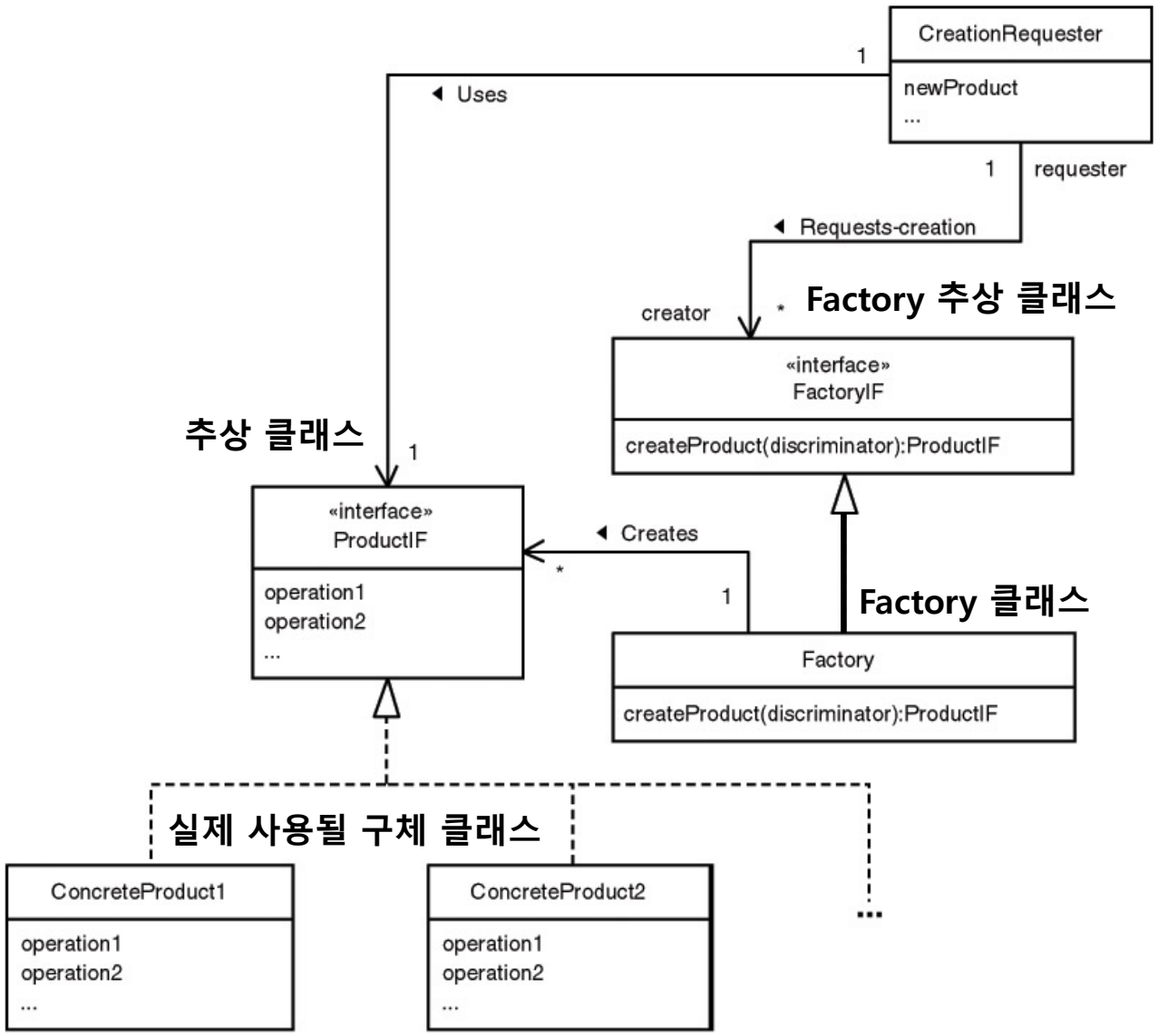
Factory Method



Factory Method (팩토리 메서드)

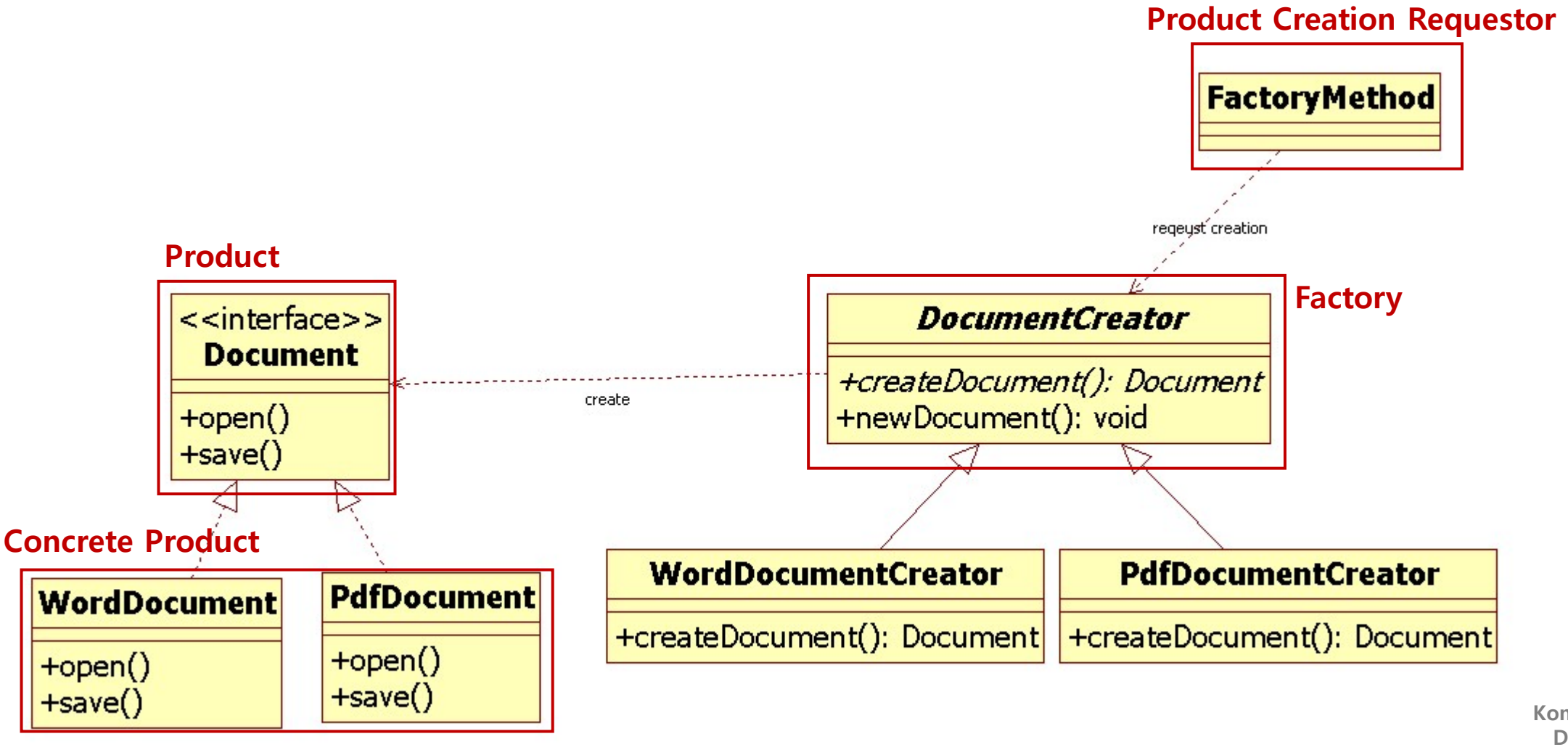
- 기본 구조



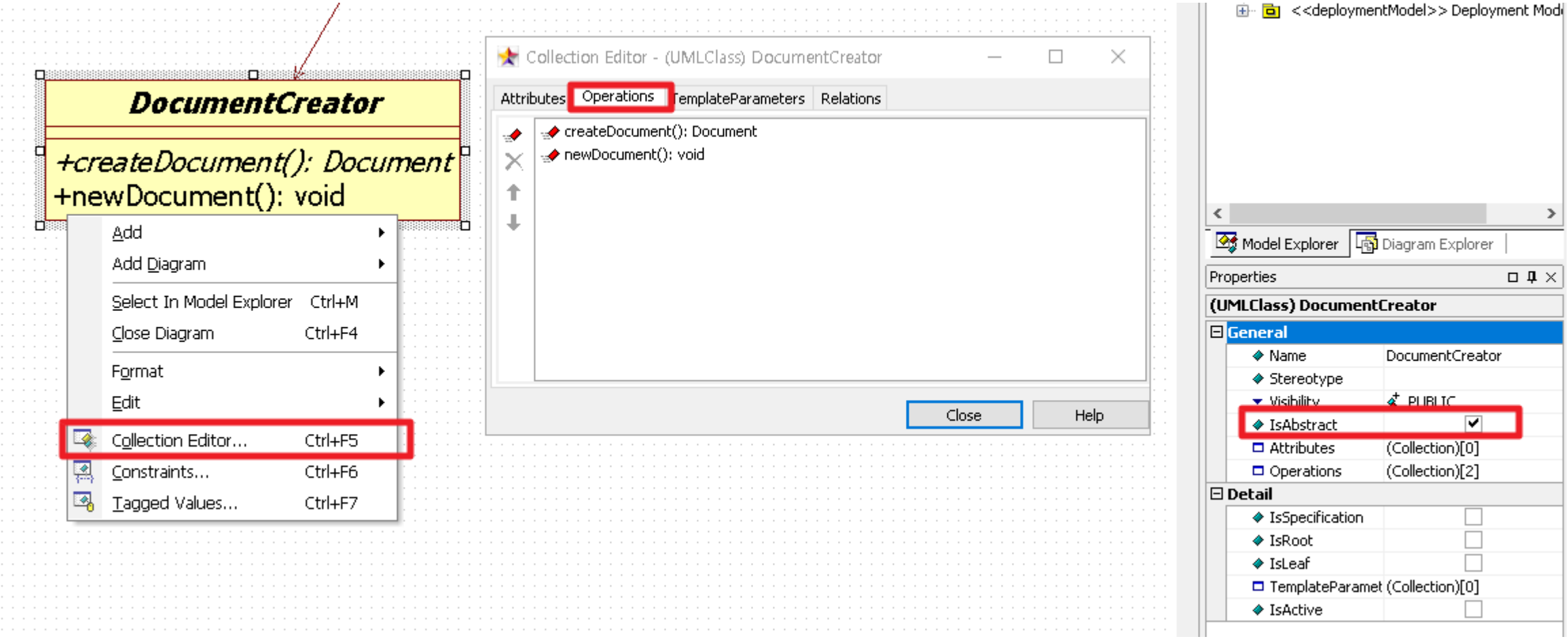


실습: FactoryMethod

- 문서 생성기



Method를 Abstract Method로 만드는 법



실습: FactoryMethod.java

```java
// ===== 1. Product (제품) =====
// 제품의 공통 인터페이스
interface Document {
    void open();
    void save();
}

// ===== 2. ConcreteProduct (실제 제품) =====
// 실제 워드 문서
class WordDocument implements Document {
    public void open() { System.out.println("Opening Word document..."); }
    public void save() { System.out.println("Saving Word document..."); }
}

// 실제 PDF 문서
class PdfDocument implements Document {
    public void open() { System.out.println("Opening PDF document..."); }
    public void save() { System.out.println("Saving PDF document..."); }
}

// ===== 3. Creator (생성자) =====
// 생성자의 공통 인터페이스 (핵심: 팩토리 메소드)
abstract class DocumentCreator {
    // 팩토리 메소드: 서브클래스가 구현해야 함
    public abstract Document createDocument();

    // 팩토리 메소드를 사용하는 공통 로직
    public void newDocument() {
        Document doc = createDocument(); // 팩토리 메소드 호출!
        doc.open();
    }
}
```

```java
// ===== 4. ConcreteCreator (실제 생성자) =====
// 워드 문서를 만드는 생성자
class WordDocumentCreator extends DocumentCreator {
    @Override
    public Document createDocument() {
        // "어떤" 문서를 만들지 서브클래스가 결정
        return new WordDocument();
    }
}

// PDF 문서를 만드는 생성자
class PdfDocumentCreator extends DocumentCreator {
    @Override
    public Document createDocument() {
        // "어떤" 문서를 만들지 서브클래스가 결정
        return new PdfDocument();
    }
}

// ===== 5. Client (사용자) =====
public class FactoryMethod {
    Run | Debug
    public static void main(String[] args) {
        // 사용자는 "어떤" 문서가 생성되는지 모름
        // 오직 "어떤 생성자"를 쓸지만 결정

        System.out.println("Client: I need a Word document.");
        DocumentCreator wordCreator = new WordDocumentCreator();
        wordCreator.newDocument(); // "Opening Word document..." 출력

        System.out.println("\nClient: Now I need a PDF document.");
        DocumentCreator pdfCreator = new PdfDocumentCreator();
        pdfCreator.newDocument(); // "Opening PDF document..." 출력
    }
}
```

실습: Factory Method

- Result

```
Client: I need a Word document.
Opening Word document...

Client: Now I need a PDF document.
Opening PDF document...
```

Design Pattern 2

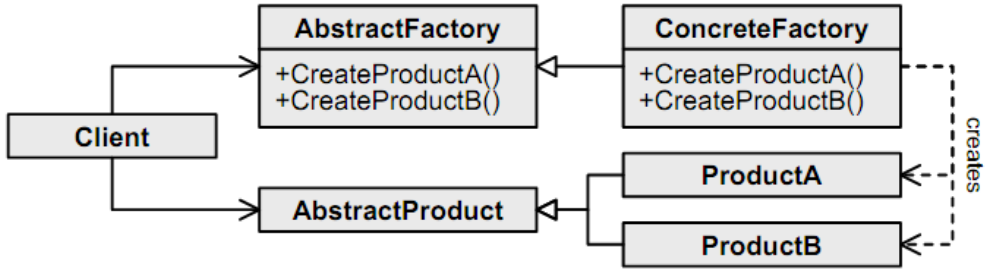
- Abstract Factory

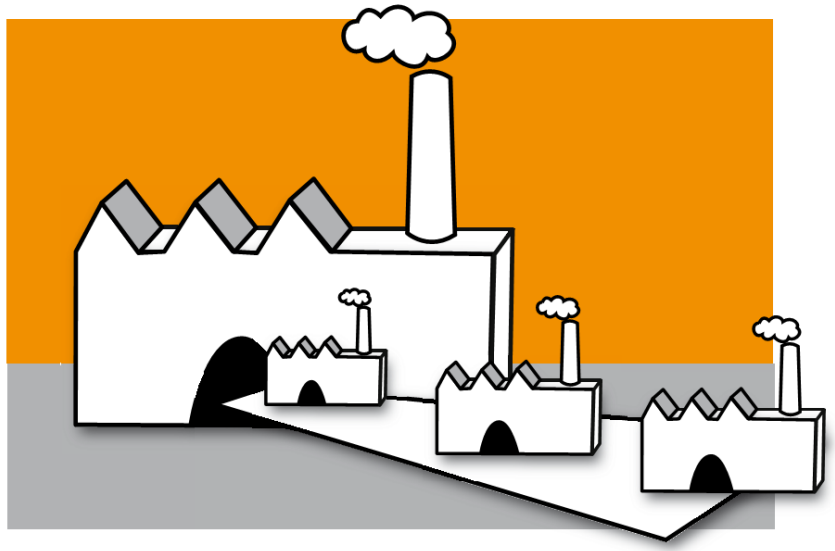
디자인 패턴 : Abstract Factory

Abstract Factory (= Kit 또는 Toolkit)

- Factory Method 패턴보다 좀 더 높은 추상화 단계이다.
- 어떤 팩토리냐에 따라 일관된 테마를 가진 객체 세트를 만들어내는 패턴
- Abstract Factory는 여러 클래스 그룹 중의 하나를 되돌려 받는 Factory 객체라고 할 수 있다.
- 다양한 구성요소 별로 '객체의 집합'을 생성할 때 유용하다. 즉, 서로 다른 객체들을 하나의 팩토리에서 생성 및 관리한다고 보면 된다.
- 하나의 추상 클래스에서 객체의 생성을 처리하고, 다양한 성격의 객체를 하나의 군으로 형성 그것을 객체단위로 취급하여 생성을 해야 할 때 유용한 패턴이다.

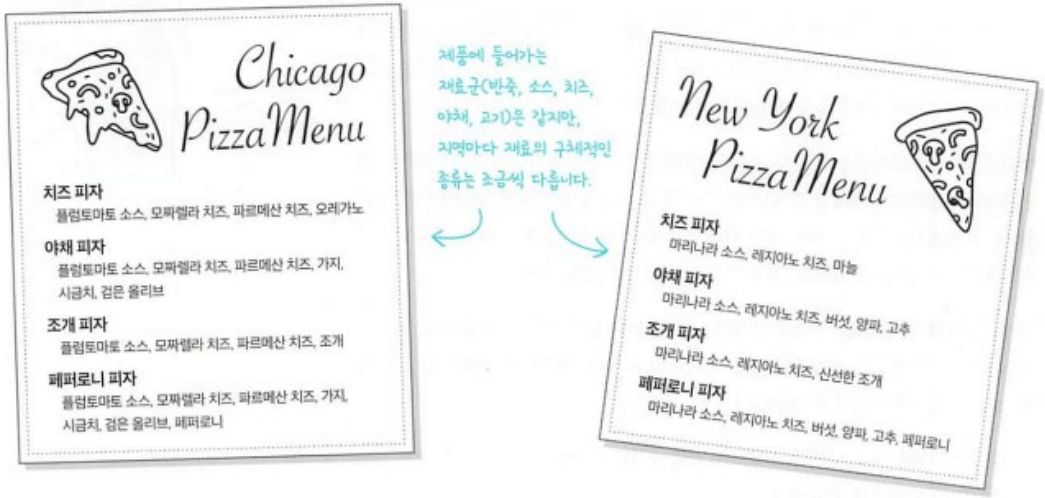
기본 클래스 다이어그램

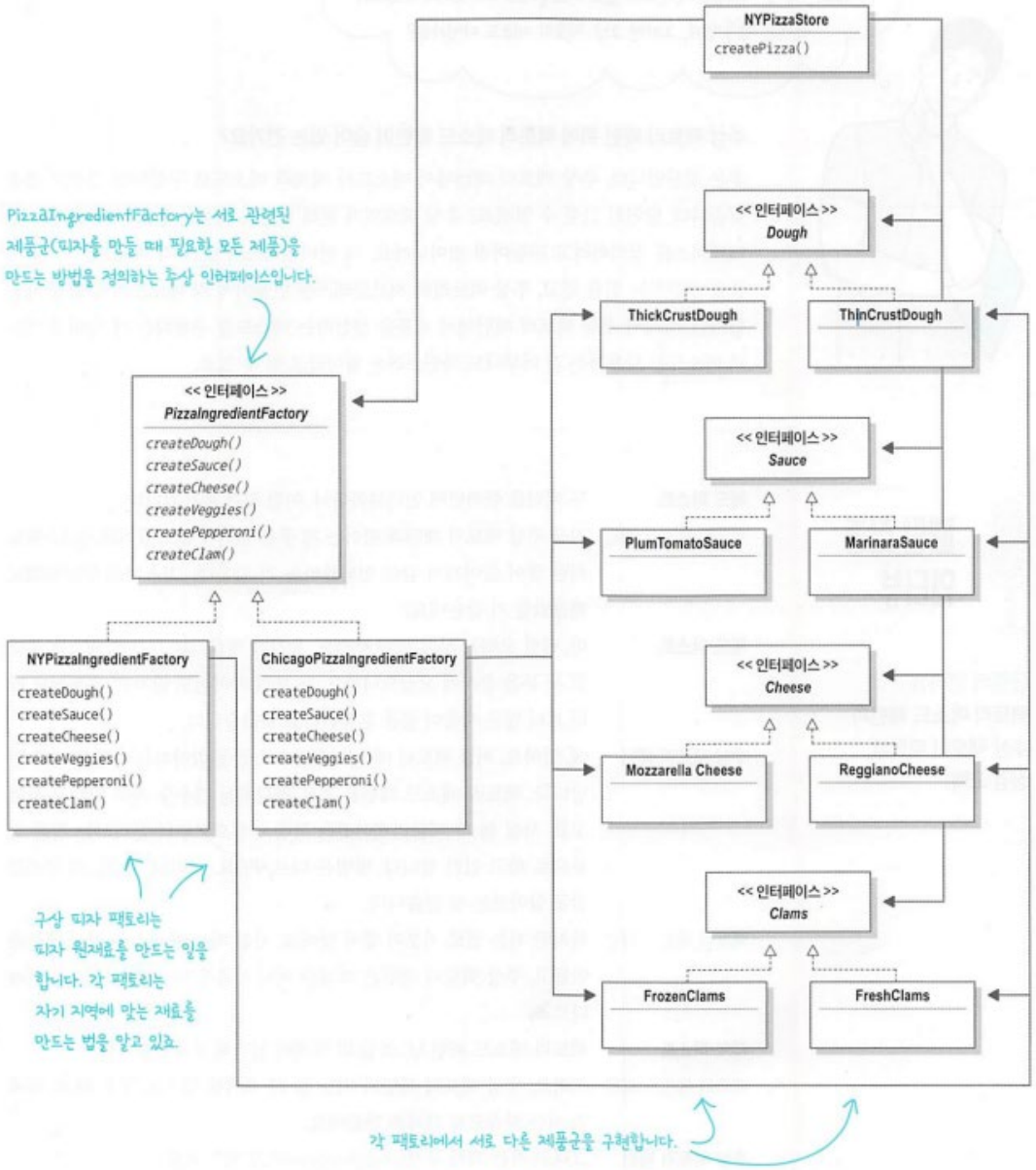




Abstract Factory (추상 팩토리)

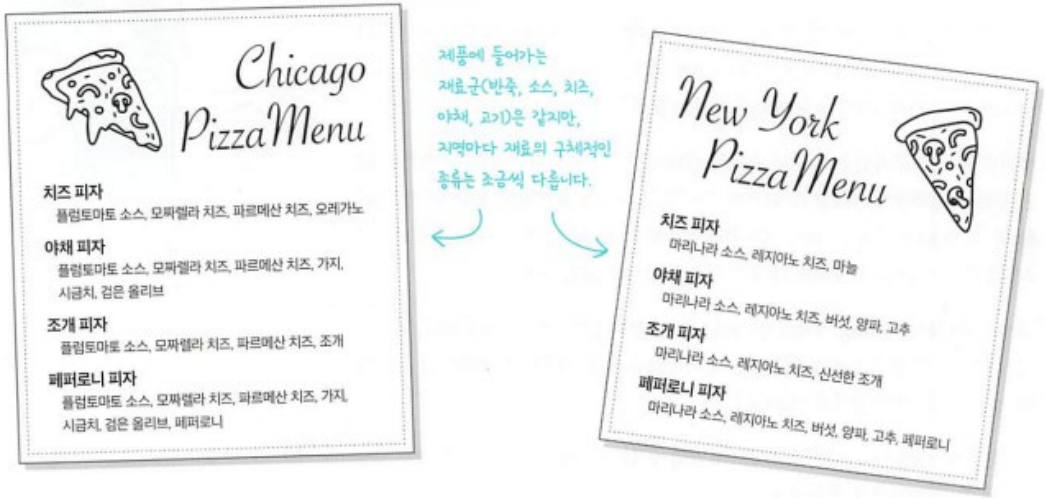
- 예시) 피자주문 – 가게마다 다른 재료?
  - 사용하는 재료군은 같지만 종류가 다른 경우
  - 종류별로 그룹을 만들어서 객체를 한번에 가져옴.





Abstract Factory (추상 팩토리)

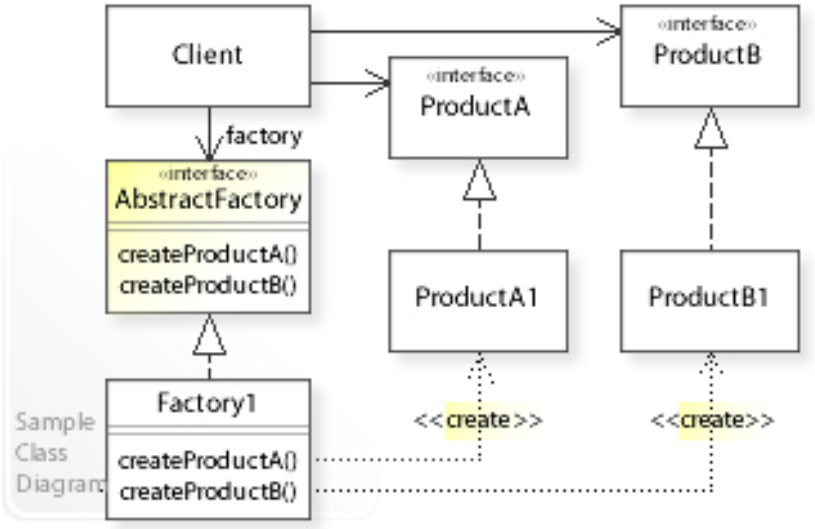
- 예시) 피자주문 – 가게마다 다른 재료?
  - 사용하는 재료군은 같지만 종류가 다른 경우
  - 종류별로 그룹을 만들어서 객체를 한번에 가져옴.

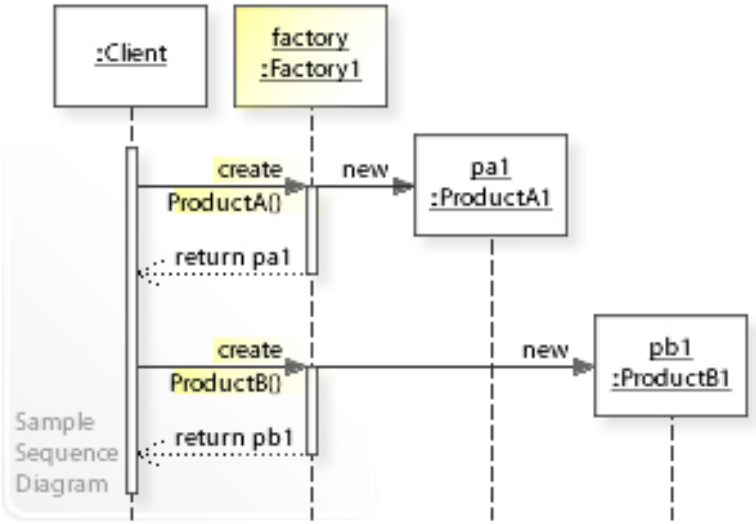



```java
public class NYPizzaStore extends PizzaStore {

    protected Pizza createPizza(String item) {
        Pizza pizza = null;
        PizzaIngredientFactory ingredientFactory =
            new NYPizzaIngredientFactory();

        if (item.equals("cheese")) {

            pizza = new CheesePizza(ingredientFactory);
            pizza.setName("뉴욕 스타일 치즈 피자");

        } else if (item.equals("veggie")) {

            pizza = new VeggiePizza(ingredientFactory);
            pizza.setName("뉴욕 스타일 야채 피자");

        } else if (item.equals("clam")) {

            pizza = new ClamPizza(ingredientFactory);
            pizza.setName("뉴욕 스타일 조개 피자");

        } else if (item.equals("pepperoni")) {

            pizza = new PepperoniPizza(ingredientFactory);
            pizza.setName("뉴욕 스타일 페퍼로니 피자");

        }
        return pizza;
    }
}
```

Abstract Factory (추상 팩토리)

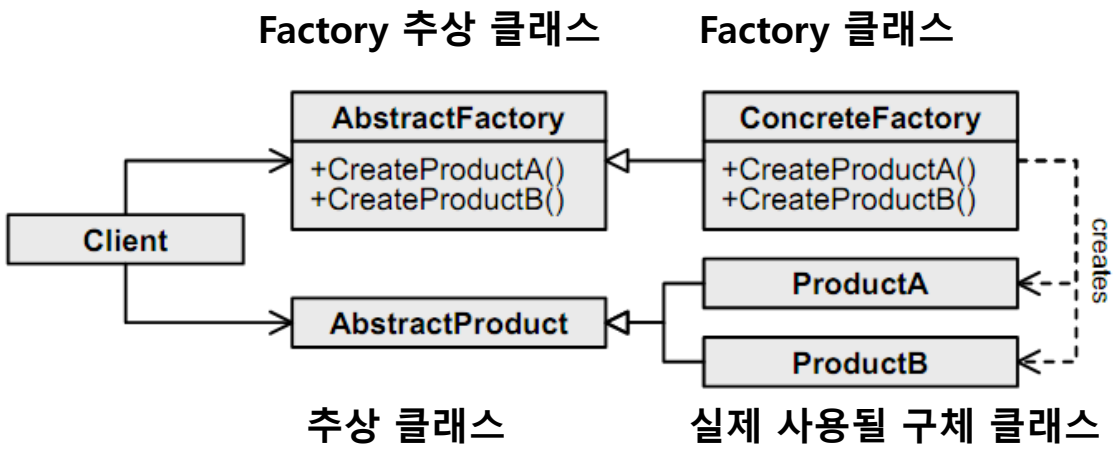
- Factory Method 패턴보다 좀 더 높은 추상화 단계
- Abstract Factory는 **여러 클래스 그룹 중의 하나**를 되돌려 받는 Factory 객체라고 할 수 있다.
- 다양한 구성요소 별로 **'객체의 집합'**을 생성할 때 유용하다. 즉, 서로 다른 객체들을 하나의 팩토리에서 생성 및 관리한다고 보면 된다.
- 하나의 추상 클래스에서 객체의 생성을 처리하고, 다양한 성격의 객체를 하나의 군으로 형성 그것을 객체단위로 취급하여 생성을 해야 할 때 유용한 패턴이다.

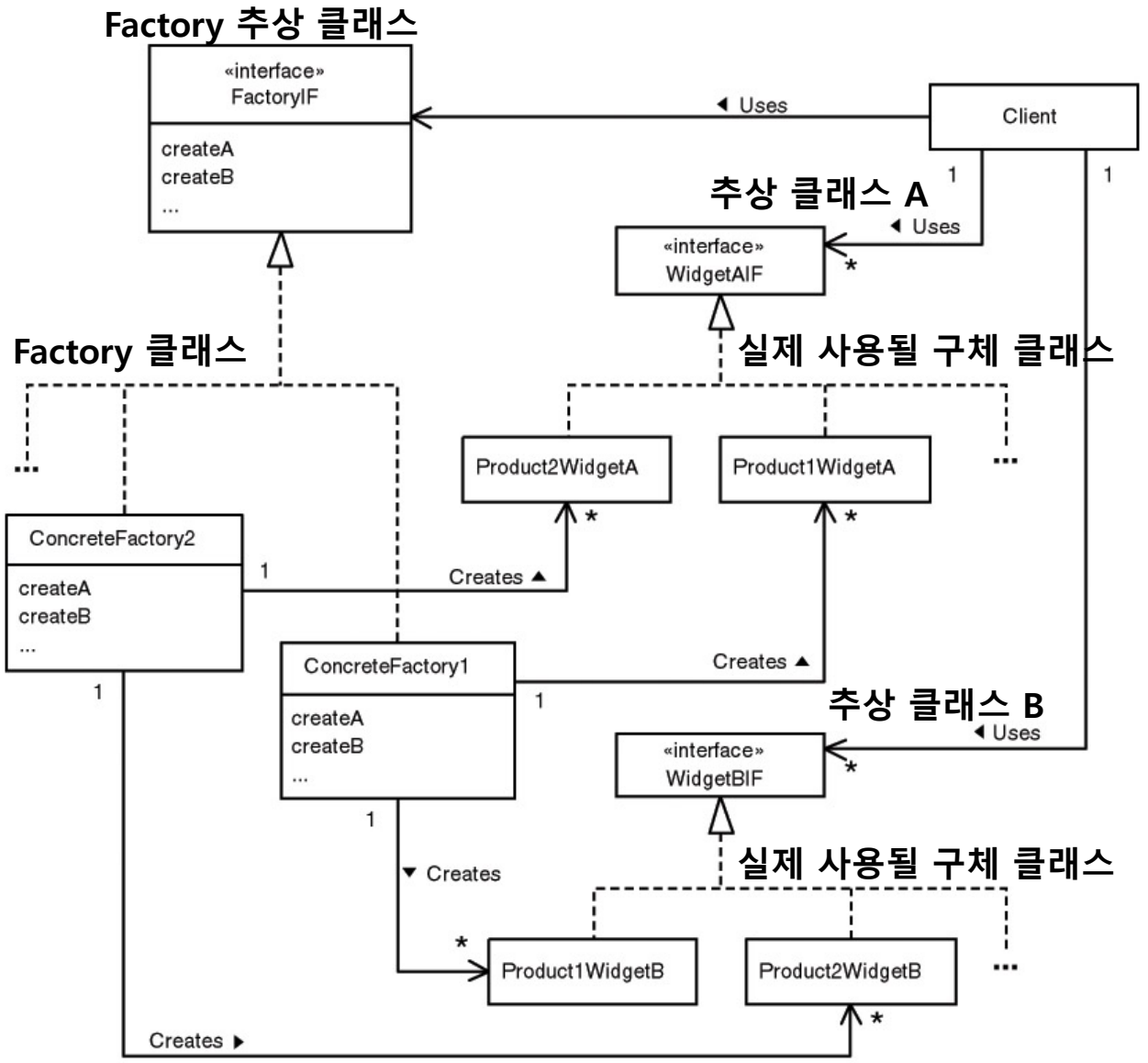




Abstract Factory (추상 팩토리)

- 기본 구조





Abstract Factory (추상 팩토리)

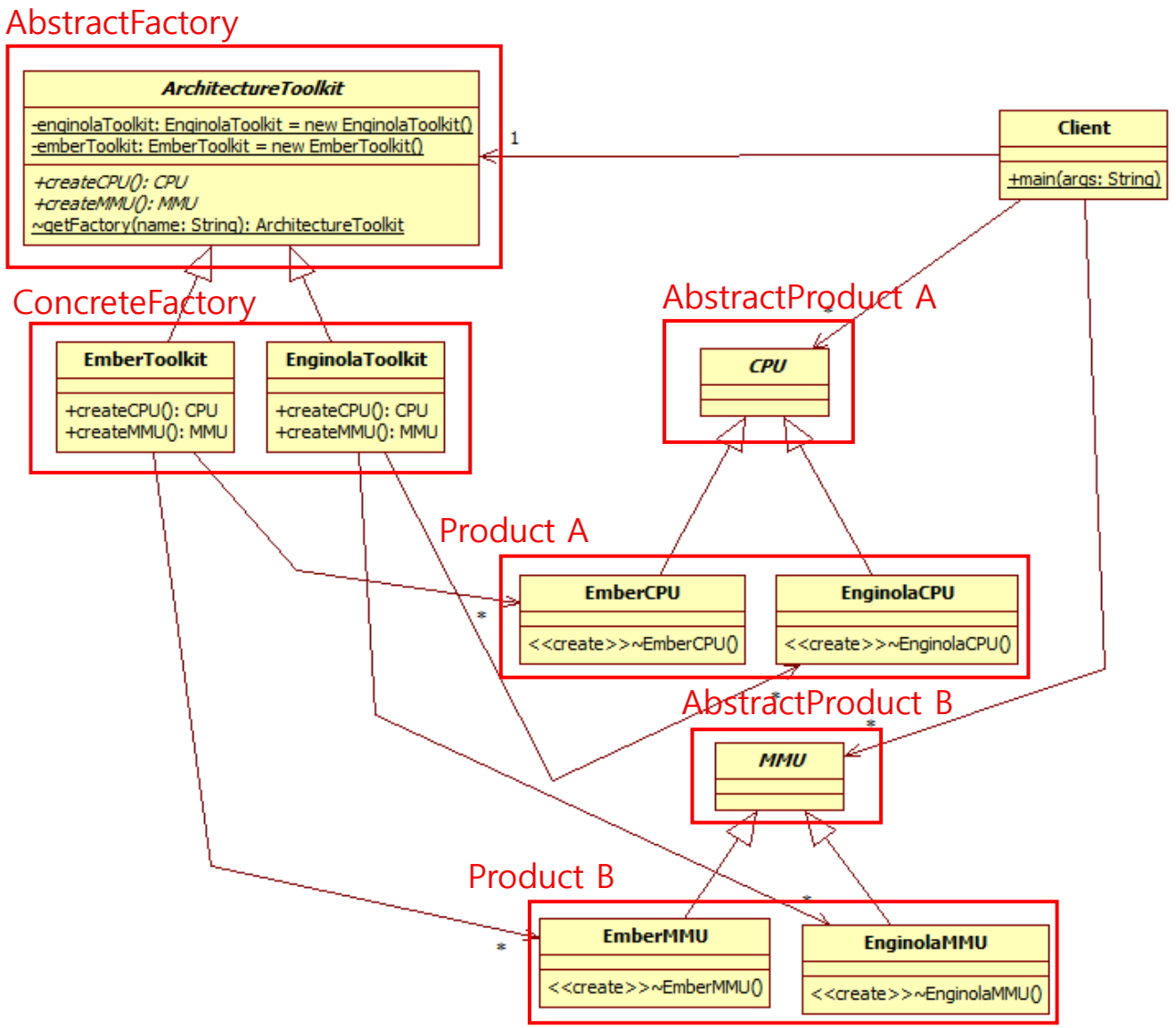
- 사용법
  - 서로 연관되거나 의존적인 객체로 이루어진 **제품군을 생산**하는 인터페이스를 제공함으로써 객체 생성을 캡슐화 하는 패턴
  - 서로 연관된 일련의 객체(제품, Product)군을 만들 때 사용
  - 생산자(Creator, Factory) 클래스는 어떤 인스턴스를 만들지 모름.

- 장점
  - 객체 구성을 활용하여 인스턴스를 생성하기 때문에, 생성되는 제품군을 쉽게 변경할 수 있음.
  - 객체를 재사용할 수 있음.

- 단점
  - 새로운 형태의 제품군을 만들 때에 인터페이스를 변경해야 함.

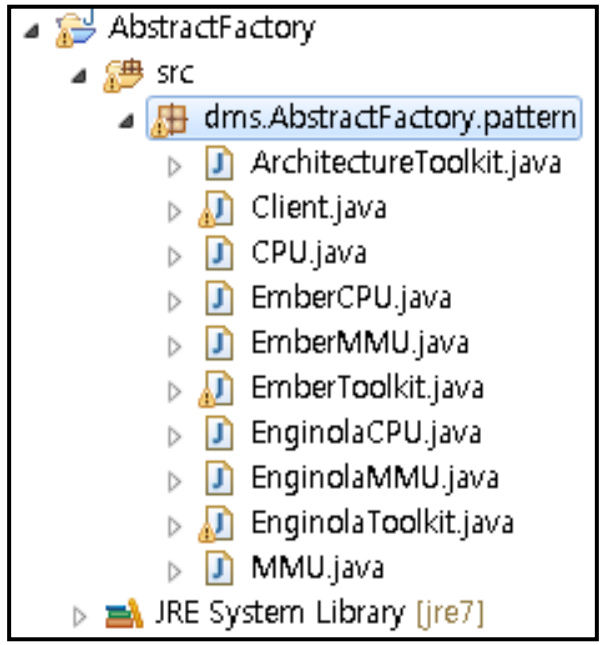
실습: Abstract Factory

- CPU/MMU



실습: Abstract Factory

- **Create JAVA Project with packages**
  - Put the generated codes into “dms.AbstractFactory.pattern” package



실습: Abstract Factory

- Client.java

```
package dms.AbstractFactory.pattern;

public class Client {

  public static void main(String args[]) {
    ArchitectureToolkit en = ArchitectureToolkit.getFactory("EN");
    ArchitectureToolkit em = ArchitectureToolkit.getFactory("EM");
    CPU cpu = en.createCPU();
    MMU mmu = em.createMMU();
  }
}
```

실습: Abstract Factory

- ArchitectureToolkit.java

```
package dms.AbstractFactory.pattern;

public abstract class ArchitectureToolkit {

  private static EnginolaToolkit enginolaToolkit = new EnginolaToolkit();
  private static EmberToolkit emberToolkit = new EmberToolkit();

  public abstract CPU createCPU();

  public abstract MMU createMMU();

  static final ArchitectureToolkit getFactory(String name) {
    if (name.equals("EN")) {
      return enginolaToolkit;
    } else if (name.equals("EM")) {
      return emberToolkit;
    }
    return null;
  }
}
```

AbstractFactory

Toolkit 선택

실습: Abstract Factory

- EmberToolkit.java, EnginolaToolkit.java

Concrete Factory
EmberToolkit & EnginolaToolkit

```java
package dms.AbstractFactory.pattern;

public class EmberToolkit extends ArchitectureToolkit {

    public CPU createCPU() {
      return new EmberCPU();
    }

    public MMU createMMU() {
      return new EmberMMU();
    }
}
```

```java
package dms.AbstractFactory.pattern;

public class EnginolaToolkit extends ArchitectureToolkit {

    public CPU createCPU() {
      return new EnginolaCPU();
    }

    public MMU createMMU() {
      return new EnginolaMMU();
    }
}
```

실습: Abstract Factory

- CPU.java / MMU.java

CPU & MMU Abstract Class

```
package dms.AbstractFactory.pattern;

public abstract class CPU {
  public class EmberCPU extends CPU{}
  public class EnginolaCPU extends CPU{}
}
```

```
package dms.AbstractFactory.pattern;

public abstract class MMU {
  public class EnginolaMMU extends MMU{}
  public class EmberMMU extends MMU{}
}
```

실습: Abstract Factory

- EmberCPU.java, EnginolaCPU.java
- EmberMMU.java, EnginolaMMU.java

Product CPU & MMU 생산

```java
package dms.AbstractFactory.pattern;

public class EmberCPU extends CPU {
  EmberCPU(){
    System.out.println("Ember CPU Create!!");
  }
}
```

```java
package dms.AbstractFactory.pattern;

public class EnginolaCPU extends CPU {
    EnginolaCPU(){
      System.out.println("Enginola CPU Create!!");
    }
}
```

```java
package dms.AbstractFactory.pattern;

public class EmberMMU extends MMU {
  EmberMMU(){
    System.out.println("Ember MMUCreate!!");
  }
}
```

```java
package dms.AbstractFactory.pattern;

public class EnginolaMMU extends MMU {
  EnginolaMMU(){
    System.out.println("Enginola MMU Create!!");
  }
}
```

실습: Abstract Factory

- Result

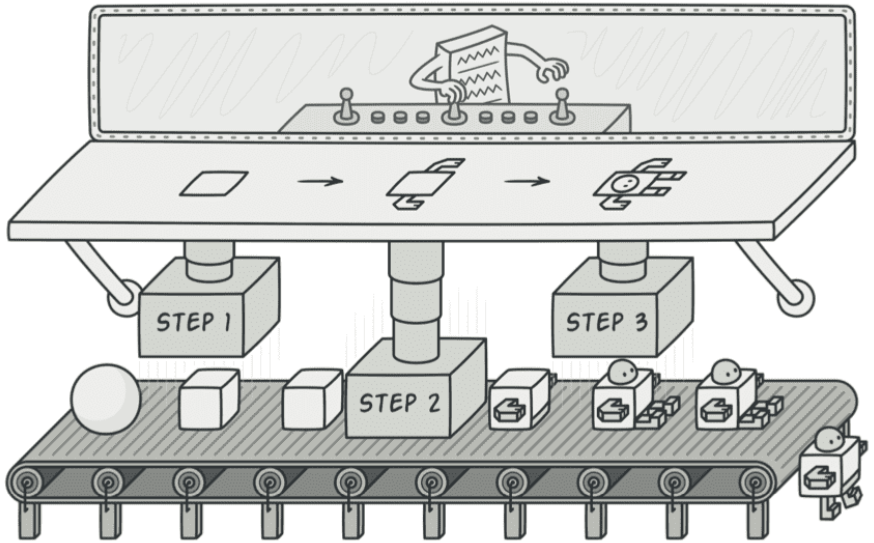
```
Enginola CPU Create!!
Ember MMUCreate!!
```

Design Pattern 2

- **Builder**

Builder (빌더)

- 복잡한 객체를 인스턴스 화(생성할 때, 메모리에 적재할 때)할 때, 생성하는 단계를 캡슐화 및 분리하여 객체의 생성을 유연하게 해주는 패턴
- 즉, **객체의 생성과정과 객체의 표현 방법**을 분리함
- 복잡한 객체들을 단계별로 생성할 수 있게 해주는 패턴



Builder (빌더)

- 사용법
  - 클래스를 설계하다 보면, attribute의 개수가 많을 경우 다음과 같은 상황이 발생할 수 있음
  - 여러 생성자 함수를 선언하고 제공하고 싶을 경우
  - 생성자에서 Argument Type이 동일할 경우 순서를 잘못 기입할 가능성 발생

- 장점
  - 필요한 데이터만 설정할 수 있음
  - 유연성을 확보할 수 있음
  - 가독성을 높일 수 있음
- 단점
  - 빌더 생성 비용이 크지는 않지만 성능에 민감한 상황에서는 문제가 될 수 있음
  - 코드가 장황해지기 때문에 매개변수가 4개 이상은 되어야 값어치를 함
  - API는 시간이 지날수록 매개변수가 많아지는 경향이 있으므로 코드가 더 커질 가능성이 있음

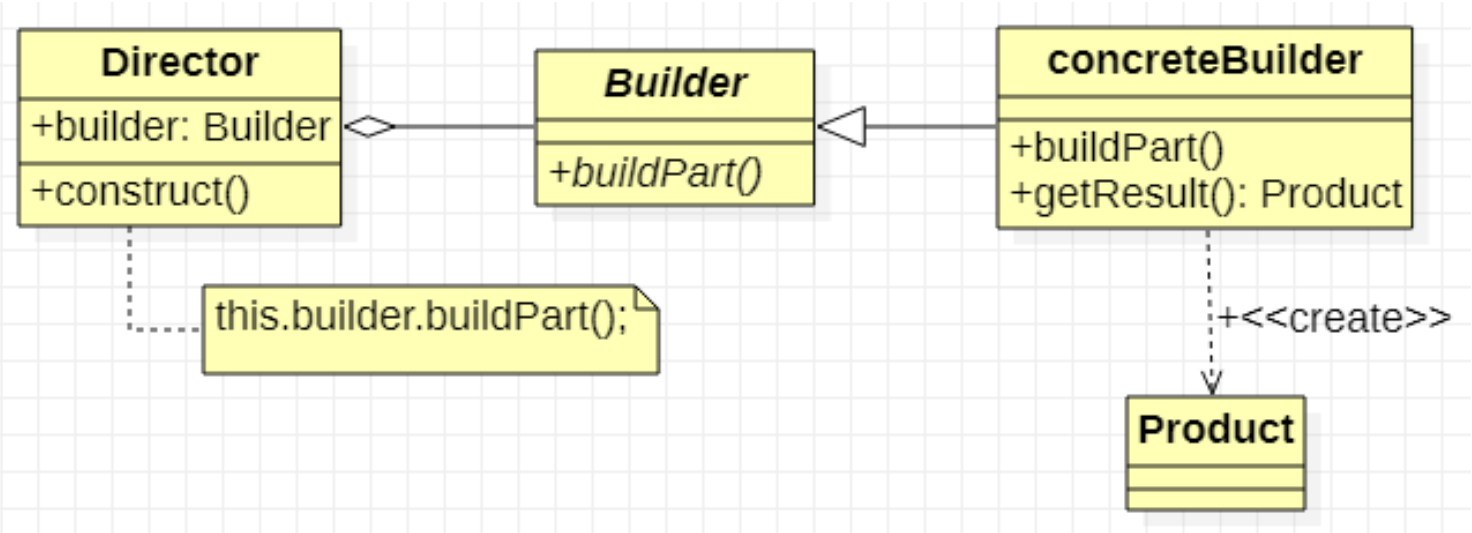
Builder Pattern을 사용할 때

- 여러 생성자 함수를 선언하고 제공하고 싶을 경우

```java
public class Hamburger {
    private String name;
    private String calories;
    private boolean tomato;
    private String beef;
    private boolean wantToEat;

    public Hamburger(final String name, final String calories) {
        this.name = name;
        this.calories = calories;
    }

    public Hamburger(final String name, final String calories, final boolean tomato) {
        this.name = name;
        this.calories = calories;
        this.tomato = tomato;
    }

    public Hamburger(final String name, final String calories, final boolean tomato,
            final String beef) {
        this.name = name;
        this.calories = calories;
        this.tomato = tomato;
        this.beef = beef;
    }
}
```

Builder (빌더)

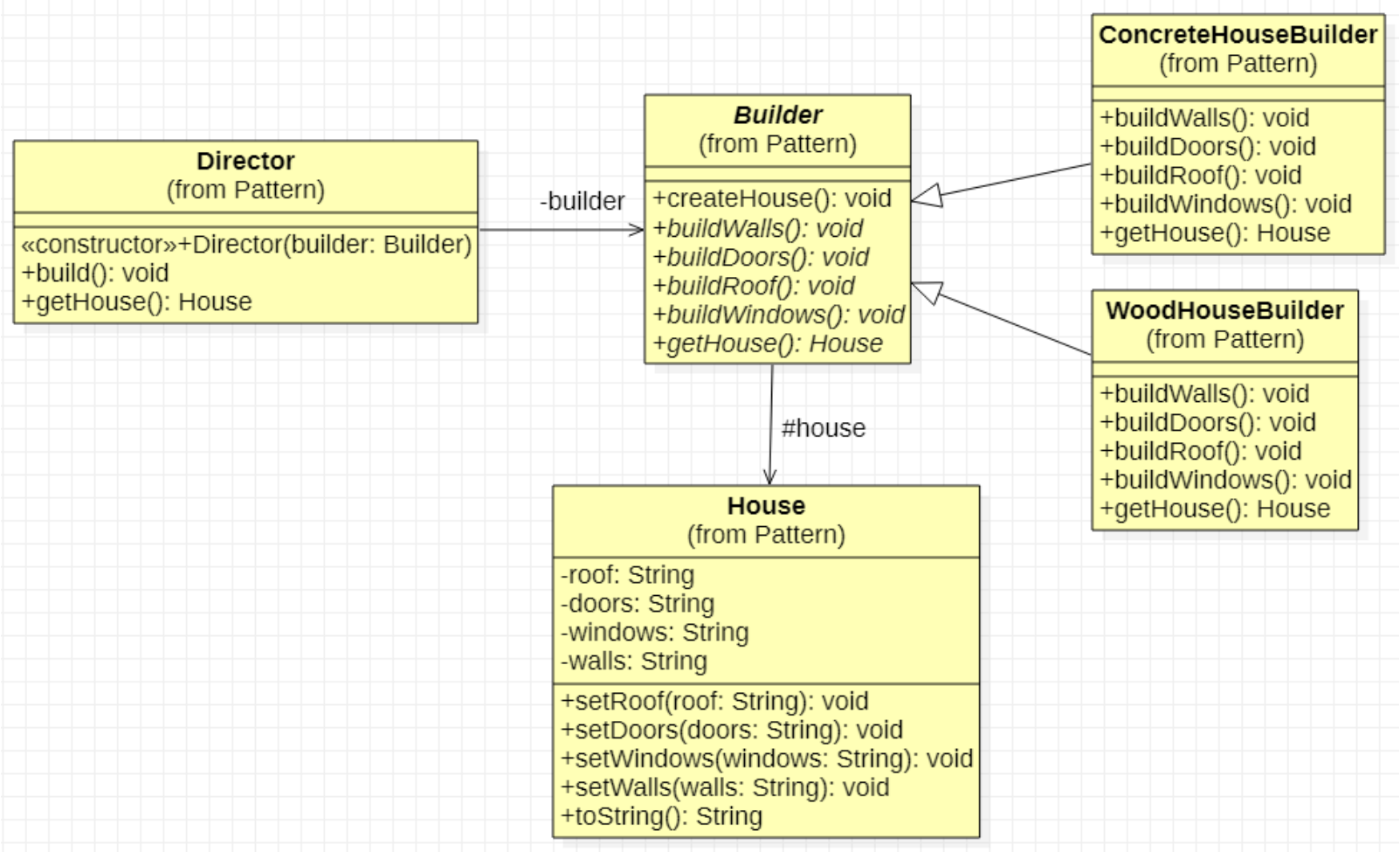
- 기본 구조



- Builder의 역할
  - 인스턴스 생성을 위한 인터페이스(API)를 선언
- ConcreteBuilder의 역할
  - Builder 인터페이스를 구현하는 역할
- Director의 역할
  - Builder 인터페이스를 사용해 인스턴스를 사용
- Product의 역할
  - 제품의 속성 및 기능을 가짐

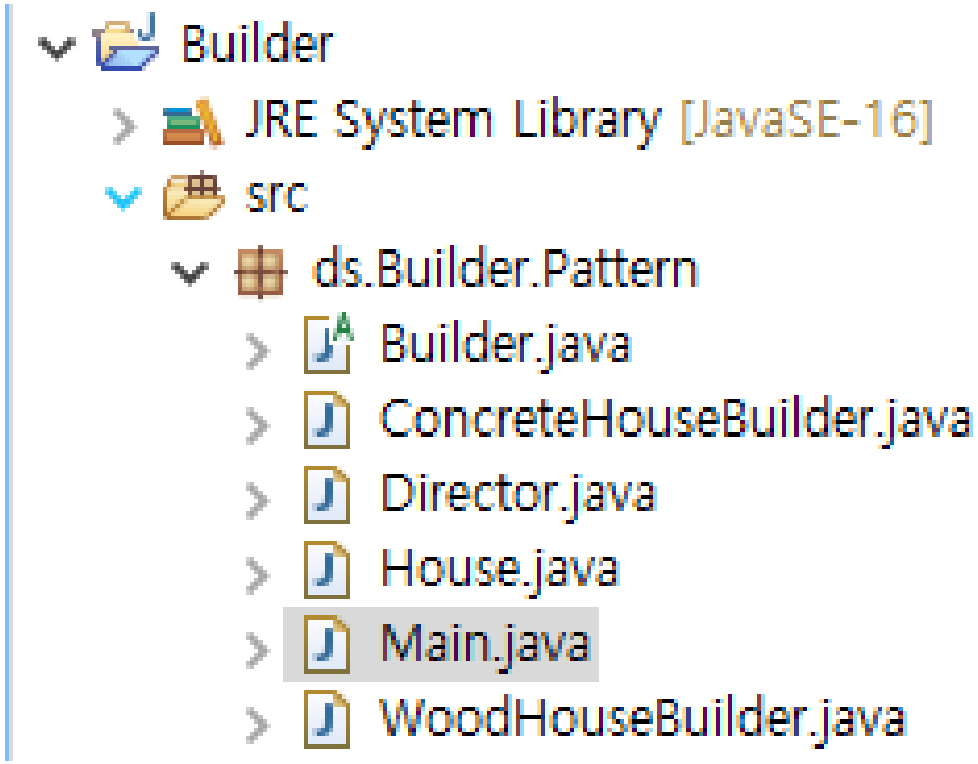
실습: Builder

- House



실습 Builder

- **Create JAVA Project with packages**
  - Put the generated codes into “ds.Builder.pattern” package



실습: Builder

- Director.java, Main.java

Builder 인터페이스를 사용해 인스턴스를 사용

```java
package ds.Builder.Pattern;

public class Director {
    private Builder builder;

    public Director(Builder builder) {
        this.builder = builder;
    }


    public void build() {
        builder.createHouse();
        builder.buildWalls();
        builder.buildWindows();
        builder.buildDoors();
        builder.buildRoof();
    }

    public House getHouse() {
        return builder.getHouse();
    }
}
```

콘크리트 집, 나무 집을 만듦

```java
package ds.Builder.Pattern;

public class Main {
    public static void main(String[] args) {
        Builder concreteHouseBuilder = new ConcreteHouseBuilder();
        Director director = new Director(concreteHouseBuilder);
        director.build();

        House house1 = director.getHouse();
        System.out.println(house1);

        Builder woodHouseBuilder = new WoodHouseBuilder();
        director = new Director(woodHouseBuilder);
        director.build();

        House house2 = director.getHouse();
        System.out.println(house2);
    }
}
```

실습: Builder

- Builder.java, House.java

인스턴스 생성을 위한 인터페이스(API)를 선언

```java
package ds.Builder.Pattern;

public abstract class Builder {
    protected House house;

    public void createHouse() {
        house = new House();
    }

    public abstract void buildWalls();
    public abstract void buildDoors();
    public abstract void buildRoof();
    public abstract void buildWindows();
    public abstract House getHouse();
}
```

실제 Product

```java
package ds.Builder.Pattern;

public class House {
    private String roof;
    private String doors;
    private String windows;
    private String walls;

    public void setRoof(String roof) {
        this.roof = roof;
    }

    public void setDoors(String doors) {
        this.doors = doors;
    }

    public void setWindows(String windows) {
        this.windows = windows;
    }

    public void setWalls(String walls) {
        this.walls = walls;
    }

    @Override
    public String toString() {
        return "이집은 [" + roof + " 지붕과, " + walls +
                " 벽과, " + windows + "창문과 ," + doors
                + " 문으로 지어짐]";
    }
}
```

실습: Builder

Builder 인터페이스를 구현하는 역할

- WoodHouseBuilder.java, Concrete HouseBuilder.java

```java
package ds.Builder.Pattern;

public class WoodHouseBuilder extends Builder{
    @Override
    public void buildWalls() {
        house.setWalls("통나무");
    }

    @Override
    public void buildDoors() {
        house.setDoors("참나무");
    }

    @Override
    public void buildRoof() {
        house.setRoof("물푸레나무");
    }

    @Override
    public void buildWindows() {
        house.setWindows("티크나무");
    }


    @Override
    public House getHouse() {
        return house;
    }

}
```

```java
package ds.Builder.Pattern;

public class ConcreteHouseBuilder extends Builder {
    @Override
    public void buildWalls() {
        house.setWalls("콘크리트");
    }

    @Override
    public void buildDoors() {
        house.setDoors("철제");
    }

    @Override
    public void buildRoof() {
        house.setRoof("일반");
    }

    @Override
    public void buildWindows() {
        house.setWindows("일반");
    }

    @Override
    public House getHouse() {
        return house;
    }
}
```

실습: Builder

- Result

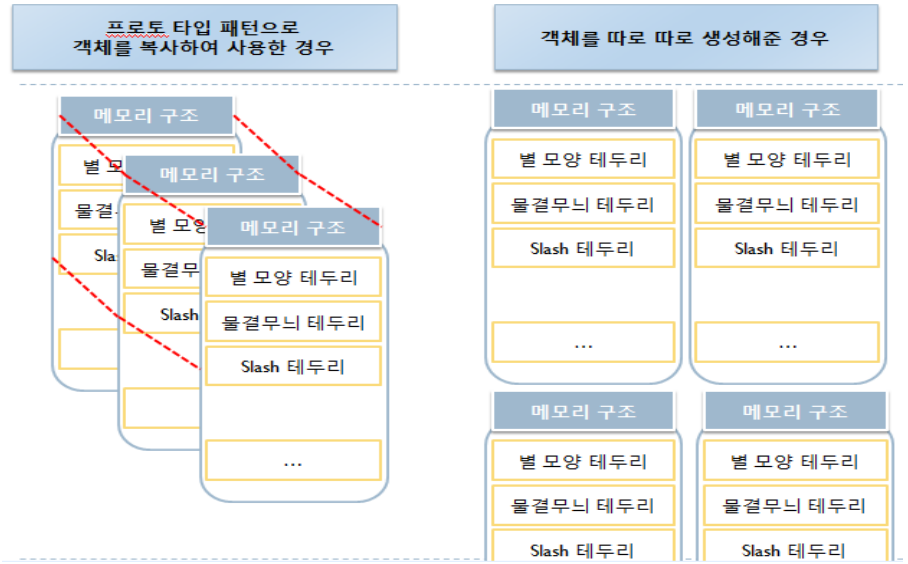
```
이집은 [일반 지붕과, 콘크리트 벽과, 일반창문과 ,철제 문으로 지어짐]
이집은 [물푸레나무 지붕과, 통나무 벽과, 티크나무창문과 ,참나무 문으로 지어짐]
```

Design Pattern 2

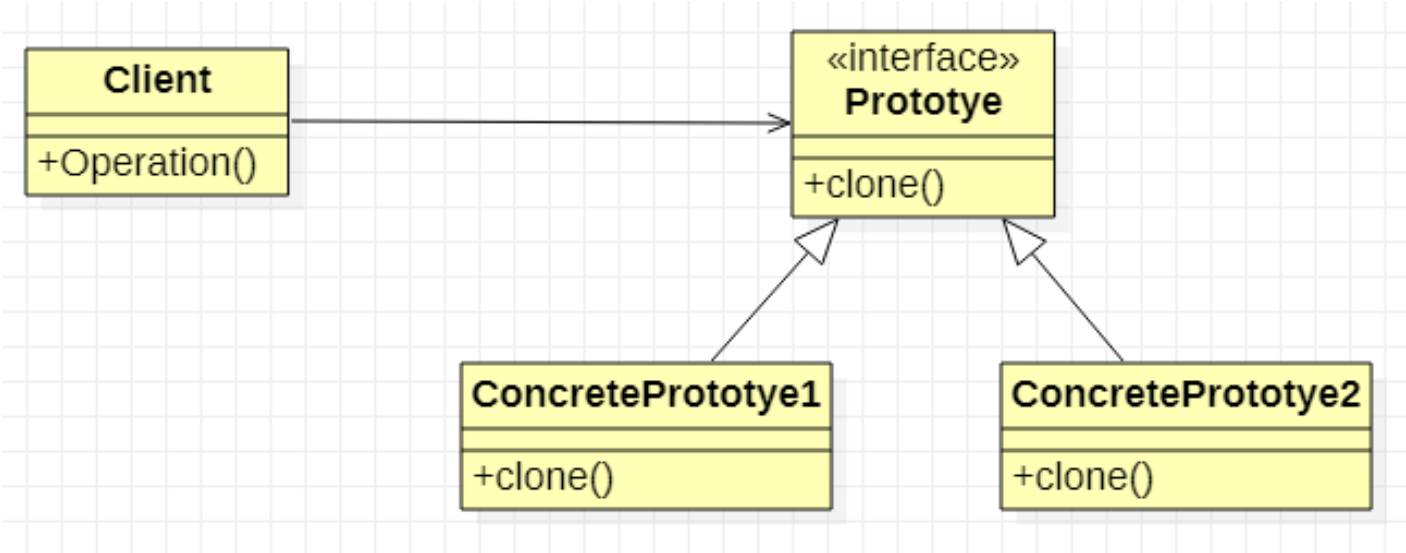
- **Prototype**

Prototype (프로토타입)

- 객체가 필요할 때마다 New 를 생성해서 객체를 생성해 사용함. 하지만 객체를 생성할 때 드는 비용이 컴퓨터 상에서는 만만치 않을 뿐더러, 너무 많은 클래스를 관리하다 보면 서브 클래스를 줄이거나, 객체의 형태나 표현방식과는 무관하게 생성 하고 싶을 때가 있음
- 이를 위해 나온 것이 바로 프로토 타입 패턴임
- 글씨에 테두리 모양을 넣는 객체를 생성한다고 하자. 우리는 이 글씨에 테두리를 넣을 때마다 테두리 객체를 생성하여 넣어줘야 할 것이다. 하지만 프로토타입 패턴을 이용하면 하나의 객체만을 생성하고 그 객체를 복사하여 쓸 수 있다



Prototype (프로토타입)



- Client의 역할
  - 프로토타입을 통해 복제를 원하는 객체를 복제
- Prototype Interface의 역할
  - 자기 자신을 복제하는 인터페이스를 구현
- ConcretePrototype1, 2의 역할
  - 자신을 복제하는 코드를 구현함

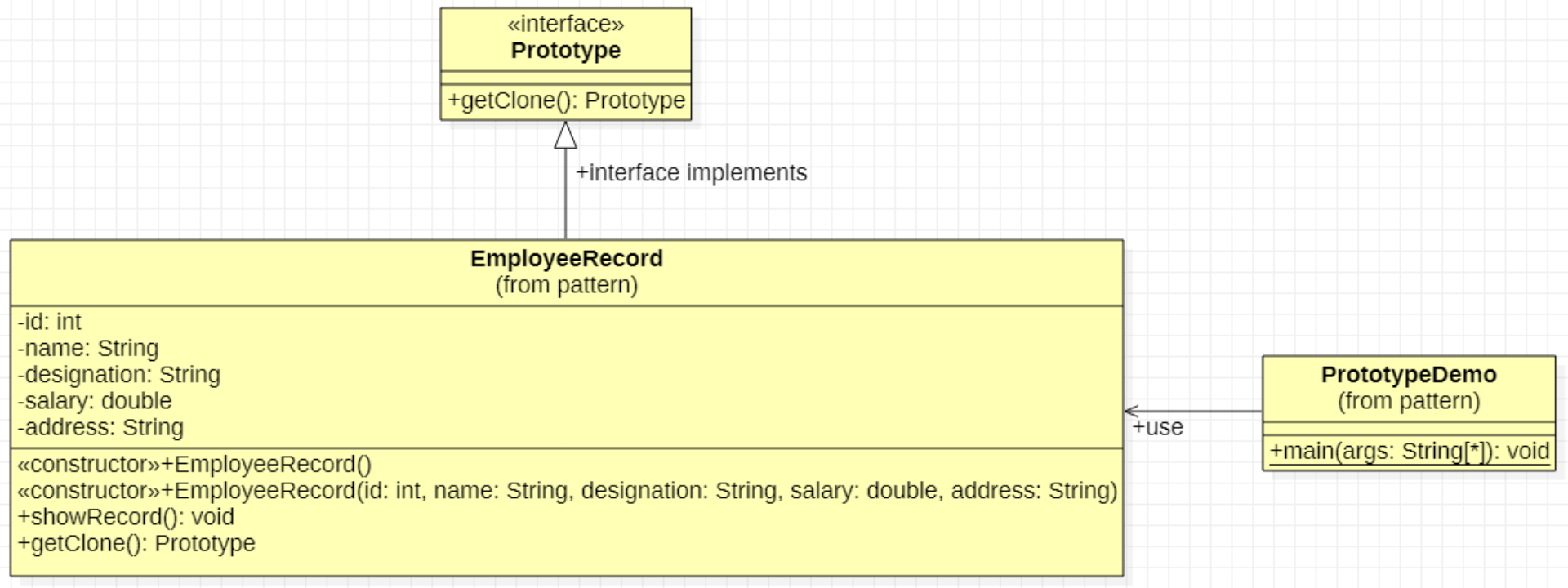
Prototype (프로토타입)

- 사용법
  - 클래스가 런타임에 인스턴스화 되는 경우
  - 객체 생성비용이 비싸거나 복잡한 경우
  - 응용 프로그램의 클래스 수를 최소로 유지하려는 경우
  - 클라이언트 응용프로그램이 객체 생성 및 표현을 인식하지 못하는 경우

- 장점
  - 서브 클래싱의 필요성을 줄임
  - 객체 생성의 복잡성을 숨길 수 있음
  - 클라이언트는 어떤 유형의 객체인지 모른 채 새로운 객체를 얻음
  - 런타임에 객체를 추가하거나 제거할 수 있음

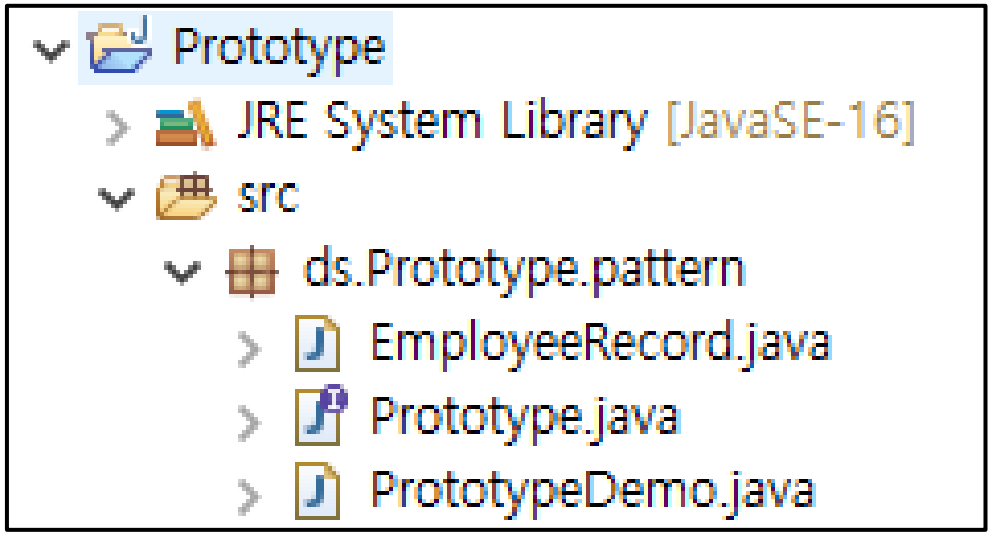
- 단점
  - 생성될 객체들의 자료형인 클래스들이 모두 clone() 메서드를 구현
  - 객체 초기화가 필수적임(팩토리 패턴과 다른점)

실습: Prototype



실습: Prototype

- **Create JAVA Project with packages**
  - Put the generated codes into "ds.Prototype.pattern" package



실습: Prototype

Prototype 인터페이스

```java
package ds.Prototype.pattern;

public interface Prototype{
    public Prototype getClone();
}
```

실습: Prototype

```java
package ds.Prototype.pattern;

public class EmployeeRecord implements Prototype{
    private int id;
    private String name;
    private String designation;
    private double salary;
    private String address;

    public EmployeeRecord(){  }

    public EmployeeRecord(int id, String name, String designation, double salary, String address){
        this();
        this.id = id;
        this.name = name;
        this.designation = designation;
        this.salary = salary;
        this.address = address;
    }

    public void showRecord(){
        System.out.println(id+"\t"+name+"\t"+designation+"\t"+salary+"\t"+address);
    }

    @Override
    public Prototype getClone(){
        return new EmployeeRecord(id,name,designation,salary,address);
    }
}
```

객체를 생성하여 반환해줌
즉, 복사한 효과

실습: Prototype

```java
package ds.Prototype.pattern;

import java.io.IOException;

public class PrototypeDemo{
    public static void main(String[] args) throws IOException {
        int eid = 100301;
        String ename = "홍길동";
        String edesignation = "직함";
        double esalary = 2000000;
        String eaddress = "서울시 서초구";

        EmployeeRecord e1=new EmployeeRecord(eid,ename,edesignation,esalary,eaddress);
        e1.showRecord();

        System.out.println("\n");

        EmployeeRecord e2=(EmployeeRecord)e1.getClone();
        e2.showRecord();
    }
}
```

e1에 데이터를 입력하여 객체 생성

e1객체를 복제함

실습: Prototype

- Result

e1객체	100301	홍길동	직함	2000000.0	서울시 서초구
e2객체	100301	홍길동	직함	2000000.0	서울시 서초구

생성패턴 5가지에 대해

- 객체를 생성(만드는)하는 패턴
  - Singleton: 자동차 공장의 “중앙 통제실”. 시스템에서 단 하나만 존재할 경우
  - Factory Method: 어떤 자동차를 만드는지는 서브클래스에게 위임
  - Abstract Factory: 자동차 풀 세트를 테마에 맞춰서 생산함
  - Builder: Client가 단계별로 하나하나 선택하여 생산(조립)함
  - Prototype: 이미 만들어진 것을 복사해서 사용

- 성능적인 측면?
  - 설계 패턴은 대부분 유연성, 유지보수성, 가독성을 위해 사용함
  - 프로토타입은 객체 생성이 매우 비쌀경우 clone() 메서드를 사용하여 복사가 가능함
  - 싱글톤은 단 하나의 객체만 존재함을 보장하므로 메모리를 절약할 수 있음
  - 빌더는 성능을 고려하지 않음, 객체 하나를 만들기 위해 builder와 car를 또 만들어야함.
  - 대부분 유연성과 low coupling을 목적으로 두고, Trade off가 가능함

실습 제출

- 아래 실습 후 모든 파일을 압축
  - Factory Method, Abstract Factory, Builder, Prototype

- 위 실습은 똑같이 따라 할 필요 없음
  - 클래스명이나 함수명등을 임의로 바꿔도 무관함
  - 클래스 다이어그램하고 PPT내의 Source Code가 상이할수 있음

- 제출할 것(학번-이름-Practice_10주차.zip)
  - 아래 파일들을 압축해서 업로드(실습 때 했던 것을 모두 압축하기)
    - 예)
      - Diagram 스크린샷
      - UML 파일
      - 소스코드

Thank you!

Any Questions?